Postgresql: транзакции

Для студентов ССА 3 курса

Определения и термины

Транзакция в SQL — это логическая единица работы, представляющая собой последовательность одного или нескольких операторов SQL, которая выполняется по принципу «всё или ничего».

Транзакции работают по принципу ACID

Принципы ACID

Atomicity (Атомарность) - транзакция либо выполняется полностью, либо не выполняется вовсе. Для этого начало транзакции отмечается командой BEGIN, а конец — либо COMMIT (фиксация изменений), либо ROLLBACK (отмена изменений).

Consistency (Согласованность) - транзакция переводит базу данных из одного согласованного состояния в другое согласованное состояние (согласованность - соблюдение установленных ограничений).

Isolation (Изоляция) - транзакции, выполняющиеся одновременно с текущей, не должны оказывать на нее влияния.

Durability (Долговечность) - после того, как данные зафиксированы, они не должны потеряться даже в случае сбоя.

Ситуация 1

Алексей первым нажал кнопку «Купить». Сервер магазина немедленно открыл транзакцию, чтобы зарезервировать товар.

BEGIN;

SELECT stock_count FROM products WHERE product_id = 101 FOR UPDATE;

-- Результат: 1 (Ноутбук есть, строка заблокирована для других)

Убедившись, что товар в наличии, система попыталась списать деньги со счета Алексея **(user_id = 1).** Но тут выяснилось неприятное: на балансе было всего 45 000 рублей. Сработало ограничение базы данных **CHECK (balance >= 0),** и запрос вернул ошибку.

```
transaction=# BEGIN;
BEGIN
transaction=*# UPDATE products SET stock_count = stock_count - 1 WHERE product_id = 101;
UPDATE 1
transaction=*# UPDATE consumers SET wallet = wallet - 50000 WHERE consumer_id = 1;
ERROR:  new row for relation "consumers" violates check constraint "consumers_wallet_check"
DETAIL:  Failing row contains (1, -5000).
transaction=!# COMMIT;
ROLLBACK
transaction=#
```

Поскольку условие «списания денег» не выполнилось, вся цепочка действий стала бессмысленной. Магазин не может отдать товар, не получив оплату. Система приняла решение отменить всё, что успела сделать в рамках этой попытки.

ROLLBACK;

-- Итог: Ноутбук вернулся на склад **(stock_count снова 1),** блокировка снята. Алексей остался без покупки.

Ситуация 2

Пока Алексей разбирался со своим банком, в магазин зашел Дмитрий. Он тоже хотел этот ноутбук. Его клиент открыл свою транзакцию и попытался захватить тот же самый товар.

-- Сессия Дмитрия

BEGIN;

SELECT stock_count FROM products WHERE product_id = 101 FOR UPDATE;

Но в ту же миллисекунду свои действия начал Сергей (**user_id = 3**). Его транзакция оказалась чуть быстрее или просто удачнее по таймингу сервера. Сергей успешно заблокировал строку с товаром.

-- Сессия Сергея

BEGIN;

SELECT stock_count FROM products WHERE product_id = 101 FOR UPDATE;

-- Успех! Строка захвачена Сергеем.

Дмитрий теперь вынужден ждать. Его запрос SELECT ... FOR UPDATE завис в состоянии ожидания, пока Сергей не закончит свои дела. Это и есть Изоляция: Дмитрий не может даже посмотреть актуальное состояние товара, пока Сергей владеет блокировкой.

Сергей прошел все проверки. Деньги на его счету были, товар был зарезервирован. Система выполнила финальные изменения: списала товар со склада, уменьшила баланс Сергея и создала запись о заказе.

UPDATE products SET stock_count = stock_count - 1 WHERE product_id = 101;

UPDATE users SET balance = balance - 50000 WHERE user_id = 3;

INSERT INTO orders (user_id, product_id, amount) VALUES (3, 101, 50000);

COMMIT;

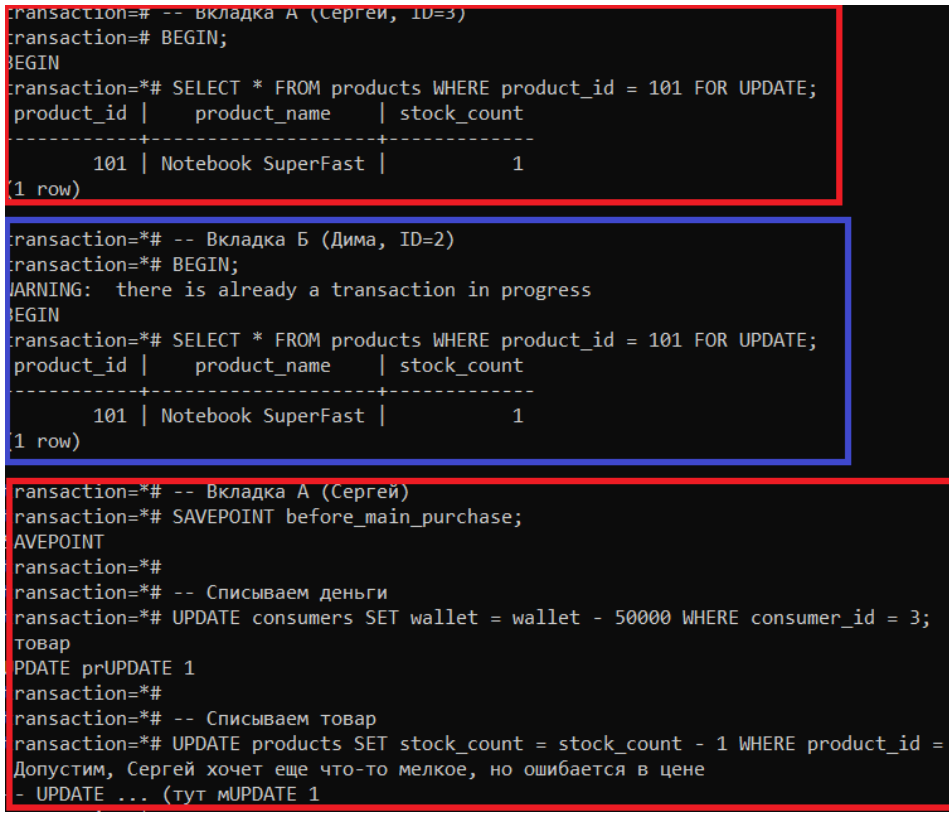
-- Итог: Данные записаны на диск. Ноутбук официально продан Сергею.

Как только Сергей сделал **COMMIT**, блокировка с товара снялась. Запрос Дмитрия, который так долго ждал, наконец-то выполнился. Но вместо ожидаемой единицы он увидел нуль.

Увидев, что **stock_count = 0**, логика приложения Дмитрия поняла: покупать нечего. Нет смысла продолжать транзакцию.

ROLLBACK;

-- Итог: Дмитрий закрыл транзакцию без изменений. Он узнал, что опоздал.



AUTOCOMMIT off

Когда вы выполняете транзакции непосредственно из консоли psql рекомендую установить настройку

\set AUTOCOMMIT off

Зачем отключать

1. Безопасность при отладке: Если ты выполняешь серию из 10 команд и на 5-й ошибся, при включенном автокоммите первые 4 команды уже сохранятся навсегда. Данные будут испорчены.
2. Полный контроль: Ты сам решаешь, где начинается и заканчивается логическая операция. База не будет «фиксировать» каждый чих без твоего разрешения.
3. Защита от случайных действий: Если ты случайно запустил DELETE FROM users;, при выключенном автокоммите у тебя есть шанс сделать ROLLBACK и спасти базу. При включенном — данные исчезнут мгновенно.

Установка значения

```
postgres=# \set AUTOCOMMIT OFF;
unrecognized value "OFF;" for "AUTOCOMMIT": Boolean expected
postgres=#
```